

Do Coverage and Mutation Scores of LLM-Generated Test Suites Correlate with Their Effectiveness? (Replicability Study)

JUNDA ZHAO, University of Toronto, Canada

SHURUI ZHOU, University of Toronto, Canada

ELDAN COHEN, University of Toronto, Canada

Recent advances in large language models (LLMs) have driven growing interest in using LLMs to automate test generation. Prior work commonly evaluates generated test suites using proxy metrics such as code coverage and mutation score. However, studies by Inozemtseva et al. and Papadakis et al. show that, for human-written tests, correlations among coverage, mutation, and real-bug detection can largely vanish once test suite size is controlled, raising concerns about the validity of evaluations based on proxy metrics. It also remains unclear whether these conclusions carry over to LLM-generated tests, given that prevailing LLM-based test-generation workflows differ substantially from traditional approaches.

In this paper, we conduct a large-scale replication study of these two prior works using a wide range of test suites generated by a diverse set of LLMs, and re-examine the relationships among coverage, mutation, and real-bug detection effectiveness. Our findings diverge substantially from prior results. We show that the usefulness of coverage and mutation is highly context-dependent: in regression-style settings where the code provided to the LLM can be reasonably assumed bug-free, these metrics can provide meaningful signals when comparing *across* models; in another common scenario where the code-under-test may already be buggy and the goal is to expose the bug within the code-under-test, they no longer serve as reliable indicators. We also find little evidence that test suite size is a dominant confounder for correlations among coverage, mutation, and real-bug detection for LLM-generated tests. Based on these findings, we discuss how to interpret results from prior studies and provide actionable guidance for evaluating LLM-based test generation.

CCS Concepts: • **Software and its engineering** → **Software testing and debugging**; **Empirical software validation**; • **Computing methodologies** → *Natural language processing*.

Additional Key Words and Phrases: Code Coverage, Mutation Testing, Test Effectiveness, Test Generation, Large Language Models, Replication Study

ACM Reference Format:

Junda Zhao, Shurui Zhou, and Eldan Cohen. 2026. Do Coverage and Mutation Scores of LLM-Generated Test Suites Correlate with Their Effectiveness? (Replicability Study). *Proc. ACM Softw. Eng.* 3, ISSTA, Article ISSTA002 (October 2026), 24 pages. <https://doi.org/10.1145/3832093>

1 Introduction

Software testing plays an indispensable role in software quality assurance, yet manually creating and maintaining extensive test suites remains resource-intensive and error-prone, motivating continued interest in automating this process. Recent advances in large language models (LLMs) have sparked growing interest in using these models to automatically generate test suites [12, 14, 45, 57, 58]. In this

Authors' Contact Information: Junda Zhao, University of Toronto, Department of Mechanical and Industrial Engineering, Toronto, Canada, junda.zhao@mail.utoronto.ca; Shurui Zhou, University of Toronto, Department of Electrical and Computer Engineering, Toronto, Canada, shurui.zhou@utoronto.ca; Eldan Cohen, University of Toronto, Department of Mechanical and Industrial Engineering, Toronto, Canada, eldan.cohen@utoronto.ca.



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2026 Copyright held by the owner/author(s).

ACM 2994-970X/2026/10-ARTISSTA002

<https://doi.org/10.1145/3832093>

line of work, the effectiveness of LLM-generated test suites is most commonly assessed using code coverage metrics (e.g., statement, branch, and modified condition coverage), alongside basic statistics such as the number of generated tests, compilation success rate, and pass rate [12, 14, 45, 57, 58]. While these metrics provide useful signals about test thoroughness, they do not directly measure whether generated tests can expose real bugs [59]. To move beyond coverage, some studies adopt mutation score as a proxy for test effectiveness, leveraging its ability to approximate fault detection without requiring manually curated real-world bugs [2, 14, 60]. However, despite mutation testing being a well-established technique, its validity as a surrogate for real-bug detection remains debated as mutation operators may not reflect the characteristics or complexity of real bugs encountered in practice [40]. Overall, direct evaluation of the core objective of testing—namely, the real-bug detection effectiveness of generated test suites—remains comparatively limited in the LLM-based test generation literature.

This naturally raises the question of whether these widely used proxy metrics are actually indicative of the *bug detection effectiveness* of LLM-generated test suites. The testing literature has extensively examined the relationships among coverage, mutation, and bug detection for human-written tests, but has reached mixed conclusions [4, 9, 17, 20, 35]. Notably, Inozemtseva et al. [30] show that the correlation between coverage and test effectiveness (using mutation as a proxy) can largely disappear once test suite size is controlled, and Papadakis et al. [40] report analogous findings for the relationship between mutation score and real-bug detection effectiveness. These results raise a key concern for current LLM-based evaluations: if coverage and mutation score do not reliably track bug detection effectiveness, then evaluating LLM-generated tests primarily through these proxies may be misleading. At the same time, whether conclusions drawn for human-written tests carry over to LLM-generated tests remains uncertain. LLM-based test generation typically uses the code-under-test as the primary prompt input [12, 14, 45, 57, 58], yielding a predominantly white-box setting that differs from that of many human-written tests, which are often derived from program requirements rather than synthesized entirely from the implementation [33]. Together, these factors call for a dedicated re-examination of how coverage and mutation relate to real-bug detection for LLM-generated test suites.

In this paper, we conduct a large-scale replication study of two prior studies by Inozemtseva et al. and Papadakis et al. to re-examine the relationships among test suite size, test coverage, mutation score, and real-bug detection effectiveness for LLM-generated test suites across 11 state-of-the-art (SOTA) LLMs. Our study focuses on LLM-generated tests, whereas prior studies primarily studied human-written tests, with Papadakis et al. also including tests generated by EvoSuite. Moreover, LLM-generated tests are commonly produced in a predominantly white-box setting, where the code-under-test is provided as the primary prompt input. This input may be either bug-free or buggy, which can substantially affect the behavior of the generated tests [29] and motivates further investigation of how this factor influences the correlations studied in prior work. Given these differences, we frame our study as a conceptual replication [10, 47] rather than a direct replication. Specifically, we revisit a similar set of research questions by examining whether the previously observed correlations among coverage, mutation, and real-bug detection effectiveness continue to hold for LLM-generated tests using both bug-free and buggy code as input, while adapting the research procedures to this new setting. We describe these methodological adaptations in Section 3.

Our results for LLM-generated tests diverge substantially from the conclusions reported in both prior studies; we summarize the main takeaways here. Although Inozemtseva et al. and Papadakis et al. question whether coverage and mutation metrics reliably indicate bug detection effectiveness, we find that these metrics can be informative when the code-under-test can be reasonably assumed bug-free, and the goal is regression testing (i.e., to catch bugs introduced by future code changes), even when test suite size is controlled. In this setting, certain coverage metrics and mutation scores

provide useful signals when comparing *across* models. However, in the more general and practically challenging setting where the correctness of the code-under-test provided to the LLM cannot be guaranteed, and the generated tests are expected to detect bugs present in that code, coverage becomes unreliable as an indicator of bug detection effectiveness, and mutation analysis is not applicable. Furthermore, unlike both prior studies, we find little evidence that the size of LLM-generated test suites (i.e., the number of tests) is a strong confounding factor in the relationships among coverage, mutation score, and real-bug detection effectiveness.

In summary, this paper makes the following contributions:

- We replicate the studies of Inozemtseva et al. and Papadakis et al. on the relationships among coverage, mutation, and bug detection effectiveness using over 100,000 test cases generated by 11 SOTA LLMs, and show that the resulting correlations for LLM-generated tests differ substantially from those reported in the original studies on human-written tests.
- We extend Inozemtseva et al.’s study beyond mutants to real-world bugs by examining how coverage correlates with real-bug detection for test suites generated from both bug-free and buggy code. This allows us to characterize how the coverage–effectiveness relationship changes in practically relevant settings where the code provided to the LLM may itself be buggy.
- Based on our findings, we discuss how to interpret coverage and mutation metrics in the LLM era and provide actionable guidance for using these metrics and for evaluating the quality of LLM-generated tests in future work.
- Following best practices for replication studies, we provide an artifact containing our implementation, generated data, and analysis scripts to support future replication and secondary studies on coverage, mutation, and bug detection effectiveness for LLM-based unit test generation.

Table 1. Studies focusing on evaluating LLMs for test generation.

Citation	Models	Datasets	Metrics	Prompt Content	Buggy Code
[51]	BART	METHOD2TEST [51], Defects4J	Correctness, Coverage	Code-under-test, Code context	✗
[45]	GPT-3.5	Public code repo, Defects4J	Correctness, Coverage	Code-under-test, Code context, Example usage	✗
[24]	GPT-3.5-turbo	Public code repo	Correctness, Coverage, Mutation	Code-under-test	✗
[43]	GPT-2	Public code repo	Correctness, Coverage, Lexical similarity	Code-under-test	✗
[50]	GPT-3	DynaMOSA [39], Defects4J	Correctness, Coverage, Bug detection, User study	Code-under-test	✗
[48]	Codex, StarCoder, GPT-3.5-turbo	HumanEval [11], SF110 [18]	Correctness, Coverage, Test style	Code-under-test, Code context	✗
[58]	GPT-3.5-turbo	CodeSearchNet	Correctness, Coverage, User study	Code-under-test, LLM-generated code intention	✗
[7]	GPT-3.5	Public code repo	Correctness, Coverage	Code-under-test	✗
[57]	6 models	Defects4J	Correctness, Coverage, Bug detection	Code-under-test, Code context	✗
[60]	CodeLlama, GPT-3.5, GPT-4	Public code repo	Correctness, Coverage, Mutation	Code-under-test, Code context	✗
[36]	GPT-3.5-turbo, GPT-4	Classes2Test [36]	Correctness, Coverage, Mutation, Test style	Code-under-test, Code context	✗
[46]	37 models	METHOD2TEST, Defects4J, ATLAS [53], Cephrot [28]	Correctness, Lexical similarity	Code-under-test	✗
[31]	10 models	TestGenEval [31]	Correctness, Coverage, Mutation	Code-under-test, Code context	✗
[52]	17 models	TestEval [52]	Correctness, Coverage	Code-under-test	✗
[2]	4 models	GitBug-Java [49]	Correctness, Coverage, Mutation, Bug detection	Code-under-test, Code context	✗
[29]	11 models	HumanEval [11], MBPP [6], APPS [26], SWE-Bench [32], BugsInPy [54]	Correctness, Coverage, Bug detection	Code-under-test, Code context, Code documentation	✓

2 Related Works

In this section, we contextualize the motivation in Section 1 by reviewing recent work on evaluating LLM-based unit test generation. We focus on studies that assess LLMs' test-generation capability in broadly applicable settings, as these works tend to yield more generalizable conclusions. We exclude methodology papers that propose specific techniques to improve LLM-based test generation, since they typically assume a predefined usage scenario and success criteria (e.g., maximizing coverage) and often evaluate in a narrower setting, making their conclusions less generalizable than those of general evaluation studies. We compile the reviewed studies based on the latest survey on LLM-based test generation [59] and summarize them in Table 1.

Across the reviewed studies, correctness metrics (statistics of tests that compile or pass) and code coverage are by far the most commonly reported outcomes. Five studies report mutation results, and four evaluate real-bug detection. We also observe that all studies use the code-under-test as the primary prompt input, consistent with a predominantly white-box test-generation paradigm. Finally, except for Huang et al. [29], no prior study considers the setting in which the input code itself may be buggy. Together, these observations motivate our investigation of whether commonly used evaluation metrics (e.g., correctness, coverage, and mutation) are indicative of bug detection effectiveness, and whether they remain informative when tests are generated from buggy inputs.

A natural question is why these studies converge on a similar set of evaluation metrics. Our review suggests that this evaluation setup can be traced to the earliest transformer-based unit test generation work [51]. Tracing further back, we also observe that the same metric choices were commonly adopted in the pre-LLM era for evaluating automated test generation tools such as EvoSuite [19, 44]. These tools are largely white-box and generate regression tests that encode the behavior of the program version given as input, typically assuming that the input code is bug-free. Under this assumption, evaluations are largely limited to proxy metrics such as coverage and mutation, since directly measuring real-bug detection on the code-under-test is not meaningful when that code is presumed correct.

In the LLM era, however, test generation is not inherently limited to this regression-testing paradigm. Unlike prior tools that mainly infer tests from the implementation, LLMs can, in certain cases, infer intended behavior beyond the code itself. For example, Huang et al. [29] show that LLMs can still generate bug-detecting tests from buggy code, although less frequently than from bug-free code. Hossain et al. [27] further show that additional context, such as documentation, can improve LLMs' ability to infer intended behavior from potentially buggy inputs. Together, these findings suggest that LLM-based test generation need not rely solely on the code-under-test or on the assumption that the implementation is correct. Nevertheless, most existing evaluations of LLM-based test generation largely inherit these evaluation settings from pre-LLM tools by default and, to the best of our knowledge, few studies explicitly examine what these metrics actually reflect in the LLM setting. We therefore argue that analyzing these widely adopted metrics for LLM-generated tests is necessary to interpret prior results correctly and to clarify what conclusions can (and cannot) be drawn from them.

3 Methodology

3.1 Research Questions

We structure our study around the following research questions:

- **RQ1: Is test coverage correlated with mutation score for LLM-generated test suites?** We replicate Inozemtseva et al.'s correlation analyses in the LLM setting, examining the relationship between test suite size and mutation score, coverage–mutation correlations under both unconstrained and size-controlled suites, and correlations among different coverage criteria.

- **RQ2: Is test coverage correlated with real-bug detection effectiveness for LLM-generated test suites?** We move beyond mutation to real bugs, assessing whether the correlations observed in RQ1 persist when effectiveness is measured via real-bug detection. We further consider the practically relevant setting in which the code provided to the LLM is buggy, and analyze coverage–bug detection correlations for tests generated from both bug-free and buggy code.¹
- **RQ3: Is mutation score correlated with real-bug detection effectiveness for LLM-generated test suites?** Using the data collected for RQ1 and RQ2, we replicate Papadakis et al.’s mutation–bug detection correlation analysis to evaluate whether the mutation score is predictive of real-bug detection for LLM-generated test suites.

3.2 Benchmark

In this study, we focus on large Java projects, following the scope of Inozemtseva et al. Specifically, we use Defects4J [34], a widely adopted benchmark of real bugs from open-source Java projects (e.g., JFreeChart and Apache Commons Lang) and a common choice in recent evaluations of automated and LLM-based unit test generation [3, 45, 51, 57]. Defects4J was also used by Papadakis et al. to study the relationship between mutation score and real-fault detection effectiveness. Each defect includes both a buggy version and its corresponding fixed (bug-free) version, along with developer-written tests that serve as test oracles. Defects4J v3.0 contains 854 defects across 17 projects, and each defect may modify one or more methods; we refer to these patched methods as *focal methods*. These focal methods constitute the units under test and form the core code context that we provide to LLMs for unit test generation.

Following prior work on evaluating LLM-based unit test generation [45, 57, 58], we focus test generation and evaluation on focal methods. To select focal methods, we follow the general practice of prior studies [3, 51, 57] and further refine the selection based on the following criteria:

- (1) Instead of retaining only public methods, we keep all *non-private* focal methods, since public, protected, and package-private methods can be accessed by tests placed in the same package.
- (2) Each focal method must exist in both the buggy and fixed versions to enable meaningful comparisons; we exclude cases where a focal method was removed or added after patching.
- (3) We retain only focal methods whose buggy version triggers at least one developer-written failing test in Defects4J, ensuring that the corresponding patch reflects a functional bug fix rather than a refactoring or performance-only change.

In total, our curated benchmark comprises 318 buggy focal methods spanning all 17 projects in Defects4J.

3.3 Models

To support a comprehensive and robust analysis, we evaluate 11 state-of-the-art (SOTA) large language models from six major developers, covering both *base* and *reasoning* variants where available. Reasoning models are trained to explicitly perform multi-step reasoning before producing an answer, whereas base models generate outputs directly. For hybrid models that can operate in either mode (e.g., Claude 4 Sonnet [5]), we evaluate both configurations, with reasoning enabled and disabled. Overall, this yields 13 model settings; for each setting, we generate test suites for both the buggy and fixed versions of every focal method. This breadth of models and configurations reduces the likelihood that our findings are driven by a single model family or a particular generation style,

¹We do not conduct mutation testing, or analyze the correlation between mutation score and bug detection, for buggy code in RQ2 because mutation testing presupposes a passing (“green”) test suite on the code-under-test; this would exclude precisely the tests that expose the bug within the code-under-test, hiding the very bug to be detected and rendering the resulting mutation scores meaningless. Mutation analysis is therefore not applicable in this setting.

and enables a more reliable examination of how coverage and mutation score relate to real-bug detection for LLM-generated unit tests. Table 2 presents the list of all evaluated models.

Table 2. LLMs used in our study.

Developer	Model Name	Type	Open Sourced
Google	<i>Gemini 2.5 Pro</i> [22]	Reasoning	✗
	<i>Gemini 2.5 Flash</i> [21]	Hybrid	✗
Anthropic	<i>Claude 4 Sonnet</i> [5]	Hybrid	✗
xAI	<i>Grok-4</i> [56]	Reasoning	✗
	<i>Grok-3</i> [55]	Base	✗
OpenAI	<i>GPT-4.1</i> [37]	Base	✗
	<i>GPT-O4-mini</i> [38]	Reasoning	✗
DeepSeek	<i>DeepSeek-V3</i> [16]	Base	✓
	<i>DeepSeek-R1</i> [15]	Reasoning	✓
Alibaba	<i>Qwen3-Coder-Plus</i> [41]	Base	✓
	<i>Qwen3-Plus</i> [42]	Reasoning	✓

3.4 Test Generation Workflow

To ensure a realistic and generalizable evaluation, we adopt a test-generation workflow that closely follows the comprehensive pipeline for evaluating LLM-generated unit tests proposed by Yang et al. [57], covering both prompt construction and unit test extraction. Guided by their findings on effective prompt context, we include additional, readily obtainable code features surrounding the focal method, including its parameters, the enclosing class constructor, declared fields, and other methods, as well as constructors of user-defined classes that appear as parameter types or as the return type. The focal method body is the central component of the prompt. We wrap this context with a system instruction that frames the LLM as a professional Java developer and append an explicit request to generate unit tests for the provided code. Due to space constraints, we omit the full prompt template here; it is available in our replication package.

We adopt Yang et al.’s prompt configuration for three reasons. First, Yang et al. evaluated a broad range of open- and closed-source models, supporting the generality of their prompt design. Second, they provide an extensive ablation study on the impact of additional code context, demonstrating the utility of the specific features they include. Third, the included context can be extracted solely from the code-under-test without manual inspection, enabling a fully automated prompt-construction and test-generation pipeline that aligns with the practical goal of reducing developer burden through automation.

For unit test extraction from LLMs’ outputs, we utilize an abstract syntax tree (AST) parser (Tree-sitter [8]) to extract the generated unit tests and related information, such as imports and helper functions. We then combine the extracted content with project-specific dependencies and common dependencies required by the JDK (e.g., `java.util`) and JUnit (e.g., `org.junit.Assert`), prepared in advance to avoid compilation issues due to missing dependencies.

We apply this pipeline to each focal method for each LLM, resulting in 8,268 generated test suites comprising 101,123 individual test cases.

3.5 Test Suite Construction

For test suite construction, we follow the random sampling strategy of Inozemtseva et al. and Papadakis et al. but adapt it to reflect a more realistic test suite construction process. In particular, rather than sampling individual tests from a project-wide pool, we sample at the focal-method level. We avoid project-level sampling for two reasons. First, tests are typically written to exercise different aspects of a specific focal method, so project-wide sampling can mix tests from unrelated

suites and does not reflect how test suites are constructed and used in practice. Second, when such mixed suites are size-controlled to a small number of tests, they often cover none of the mutants relevant to a given method, yielding many suites with mutation score 0 and injecting noise into the correlation analysis.

Concretely, for each draw we sample 100 focal methods, each paired with its corresponding LLM-generated test suite, from the set of generated suites that contain at least one compilable test. We construct 1,000 unique draws. In the unconstrained setting, we keep all tests in each selected suite, since LLMs naturally produce suites of varying sizes. In the size-controlled setting, because most models generate roughly 10 tests per suite on average (Figure 1), we fix suite size by selecting $k \in \{3, 5, 10\}$ and retaining k tests uniformly at random from each selected suite, restricting sampling to suites that contain at least k tests. We do not consider smaller values of k because very small suites frequently cover no mutants in the focal method, reintroducing the no-mutant-coverage issue noted above. We also avoid larger values of k because too few suites contain at least k tests, which would prevent us from constructing 1,000 unique draws.

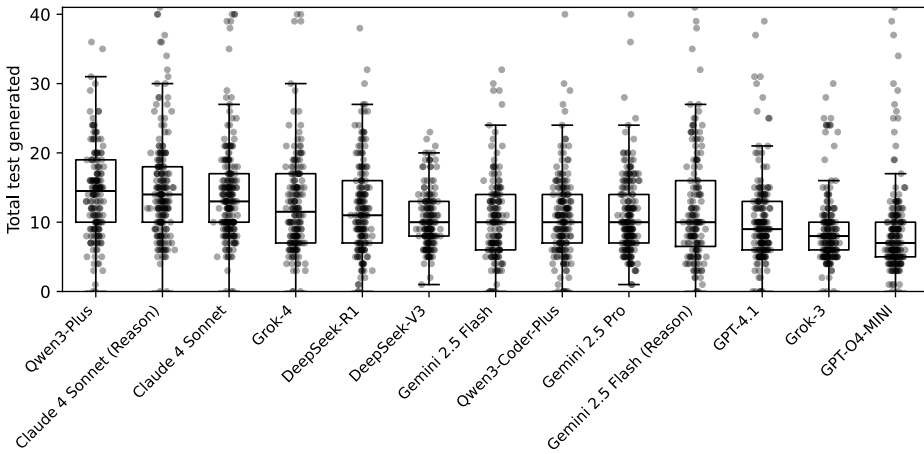


Fig. 1. Box plot of the number of tests within each test suite generated by different models.

3.6 Measurements

For the measurements required by our analysis, we largely follow the tools and metric definitions adopted by Inozemtseva et al. and Papadakis et al. We measure coverage with CodeCover [1], collecting statement, branch, and modified condition coverage (MCC). We measure mutation with PIT [13] and report both raw and normalized mutation scores using Inozemtseva et al.’s definitions, with a minor adaptation for our focal-method setting. Specifically, Inozemtseva et al. define raw mutation score as killed mutants divided by all mutants generated for the entire subject project, a ratio we instead compute over the mutants generated for the focal method associated with each test suite; their normalized mutation score, calculated as killed mutants divided by the mutants covered by the suite, is adopted unchanged. For real-bug detection, similar to Papadakis et al., we mark a buggy focal method as *detected* if its generated test suite contains at least one *effective* test that passes on the fixed version but fails on the buggy version, and we report bug detection effectiveness (the bug detection ratio) as the proportion of buggy focal methods detected.

Since our test suites are generated per focal method, we aggregate coverage and mutation in two complementary ways. In the *average* aggregation, we compute coverage (or mutation score) per focal method with respect to that method and then take the mean across focal methods. In the

accumulated aggregation, we sum covered (or killed) elements and total elements over all focal methods and compute a single global ratio. Prior work effectively reports accumulated aggregates at the project level because its analyses are based on project-wide test pools. In our focal-method setting, accumulated values can be dominated by a small number of focal methods with many statements, conditions, or mutants, potentially obscuring per-method trends. We therefore report both aggregation schemes and compute correlations only within the same aggregation type (i.e., average with average, accumulated with accumulated).

3.7 Correlation Analysis

To address our RQs, we perform three major correlation analyses: (i) coverage versus mutation score for generated test suites, (ii) coverage versus real-bug detection for test suites generated from both bug-free and buggy code, and (iii) mutation score versus real-bug detection.² As part of (i) and (ii), we also report correlations between test suite size and mutation score, and between test suite size and bug detection ratio—as in Inozemtseva et al.—to assess whether suite size confounds the relationships among the other metrics.

The large number of test suites generated by a diverse set of LLMs also enables us to examine correlations from complementary perspectives. For each correlation analysis, we report results at three granularities. (1) *Combined*: we compute correlations over sampled draws of focal-method suites pooled across all models, providing an overall view that mixes within-model and between-model variation. (2) *Intra-model*: we repeat the same analysis separately for each model to characterize how the relationship behaves among suites produced by a single LLM. (3) *Inter-model*: instead of drawing random combinations of suites as in the first two views (and in prior work), we aggregate each model’s generated test suite statistics (considering only compilable tests) into a single data point per model—for metrics such as suite size, coverage, mutation, and bug detection—and then compute correlations across models. Although this view has a smaller sample size, it can still reveal statistically meaningful patterns and help disentangle whether observed correlations are driven primarily by differences between models or by variation within a model.

4 RQ1: Correlation Between Coverage and Mutation Score

For RQ1, we replicate the study of Inozemtseva et al. by analyzing the correlation between coverage and test effectiveness (using mutation score as a proxy) for LLM-generated tests. We mirror the structure of the original work and report results for the same set of correlation analyses.

4.1 Is Size Correlated With Mutation Score?

Inozemtseva et al. reported a strong positive correlation between test suite size and both raw and normalized mutation score. To examine whether this conclusion carries over to LLM-generated tests, we conduct a similar analysis with one key difference. Rather than constructing suites of pre-specified sizes by sampling from a project-wide test pool, as in prior work, we leverage the natural variation in LLM-generated test suite sizes. Our randomly sampled LLM-generated test suites already span a wide range of test counts, allowing us to compute correlations directly between the number of generated tests and mutation score. This setup also better reflects why suite sizes vary in practice, where differences are largely driven by the characteristics and testing needs of the code-under-test rather than by manually enforcing a target size.

We begin with the combined view. As shown in Table 3, the correlations between test suite size and both raw and normalized mutation scores are uniformly weak under both average and accumulated

²We use the same Guilford scale [23] as Inozemtseva et al. for verbal descriptions: correlations with absolute value < 0.4 are “weak” (low), 0.4 – 0.7 are “moderate”, 0.7 – 0.9 are “strong”, and > 0.9 are “very strong”.

Table 3. Correlation between the number of tests (averaged for average aggregation and summed for accumulated aggregation) and both types of mutation scores over test suites from all models; all entries are significant ($p < 0.05$).

	Average aggregation		Accumulated aggregation	
	Pearson r	Kendall τ	Pearson r	Kendall τ
Raw mutation score	0.031	0.034	0.029	0.017
Normalized mutation score	0.098	0.066	-0.112	-0.047

aggregation. We then examine the same relationship from the intra-model view to assess whether the trend varies across LLMs. Figure 2 reports the within-model correlations; compared to the pooled analysis, these correlations are in many cases slightly higher but remain weak for every model. Finally, we consider the inter-model view: Table 4 shows that these model-level correlations are also weak and not statistically significant. Overall, in contrast to Inozemtseva et al., we observe weak to negligible correlations between test suite size and both raw and normalized mutation score for LLM-generated test suites across combined, intra-model, and inter-model analyses. This suggests that LLMs that generate more tests do not necessarily produce test suites with stronger mutant-killing capability.

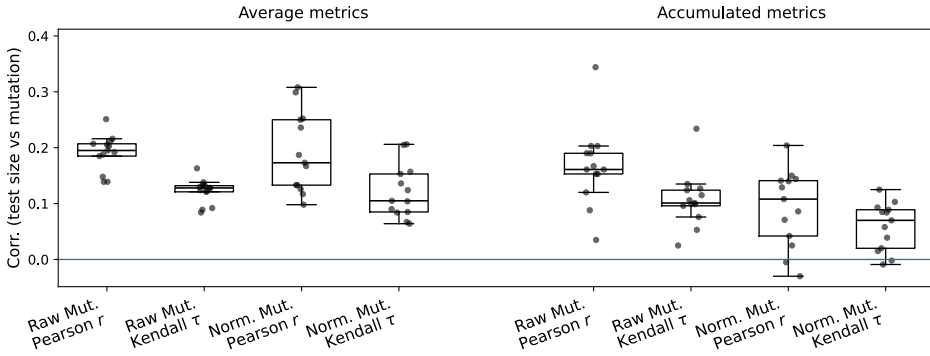


Fig. 2. Box plots of the correlation between test suite size and mutation score (raw and normalized) in the unconstrained setting. Each dot is the correlation computed from model-specific sampled test suite combinations; boxes group these correlations by aggregation type (average vs. accumulated) and statistic (Pearson r vs. Kendall τ).

Table 4. Inter-model correlation between the number of tests (averaged for average aggregation and summed for accumulated aggregation) and mutation scores; p -values are reported in parentheses.

	Average aggregation		Accumulated aggregation	
	Pearson r	Kendall τ	Pearson r	Kendall τ
Raw mutation score	0.185 (0.546)	0.205 (0.367)	0.259 (0.393)	0.231 (0.306)
Normalized mutation score	0.093 (0.761)	0.026 (0.952)	-0.347 (0.245)	-0.231 (0.306)

4.2 Is Coverage Correlated With Mutation Score When Size Is Ignored?

We next examine the relationship between coverage and mutation score without controlling for test suite size. Table 5 summarizes the combined-view results. Correlations are consistently weak across coverage types and mutation-score variants under both average and accumulated aggregation:

nearly all coefficients are below 0.4, with the only exception being the Pearson correlation between average statement coverage and average normalized mutation score (0.443), which is just above the “moderate” threshold. The same pattern holds within individual models. As shown in Figure 3, intra-model correlations are typically even weaker than those in the pooled analysis: coefficients cluster tightly around zero and, in most settings, remain below 0.2.

Table 5. Correlation between coverage and mutation score across combinations of randomly selected test suites from all models, reported for both average aggregation and accumulated aggregation; all entries are significant ($p < 0.05$), except the accumulated-aggregation correlation between normalized mutation score and statement coverage.

Coverage	Mutation score	Average aggregation		Accumulated aggregation	
		Pearson r	Kendall τ	Pearson r	Kendall τ
Statement	Raw	0.356	0.250	0.206	0.162
	Normalized	0.443	0.272	-0.009	-0.004
Branch	Raw	0.379	0.270	0.142	0.109
	Normalized	0.348	0.238	0.032	0.022
MCC	Raw	0.267	0.186	0.118	0.092
	Normalized	0.339	0.218	-0.087	-0.064

The even weaker correlations under the intra-model view suggest that the slightly stronger correlations in the combined analysis might be driven by differences between models rather than by variation within a single model. We confirm this in the inter-model analysis. Table 6 shows that, under average aggregation, correlations increase noticeably for most coverage–mutation combinations. Although this view has a small sample size (13 data points), most coefficients remain statistically significant ($p < 0.05$), indicating that models achieving higher average coverage also tend to achieve higher average mutation scores.

Table 6. Correlation between coverage and mutation score with test statistics aggregated into one data point for each model, reported for both average aggregation and accumulated aggregation when test suite size is ignored; p -values are reported in parentheses.

Coverage	Mutation score	Average aggregation		Accumulated aggregation	
		Pearson r	Kendall τ	Pearson r	Kendall τ
Statement	Raw	0.676 (0.011)	0.462 (0.030)	0.383 (0.197)	0.410 (0.057)
	Normalized	0.738 (0.004)	0.538 (0.010)	-0.047 (0.880)	-0.051 (0.858)
Branch	Raw	0.821 (0.001)	0.641 (0.002)	0.246 (0.419)	0.154 (0.510)
	Normalized	0.596 (0.032)	0.410 (0.057)	0.035 (0.909)	0.051 (0.858)
MCC	Raw	0.541 (0.057)	0.385 (0.076)	0.289 (0.339)	0.308 (0.163)
	Normalized	0.625 (0.022)	0.462 (0.030)	-0.124 (0.686)	-0.051 (0.858)

4.3 Is Coverage Correlated With Mutation Score When Size Is Fixed?

We now analyze the correlation between coverage and mutation score after controlling for test suite size. Due to space constraints, we omit the full set of coefficients for the combined and intra-model views and summarize the results here. Overall, these size-controlled results mirror the unconstrained setting: whether we pool suites across all models or analyze each model separately, the correlation between coverage and mutation score remains weak across all tested suite sizes. The more interesting question is whether the moderate-to-strong inter-model correlation observed earlier persists once suite size is fixed.

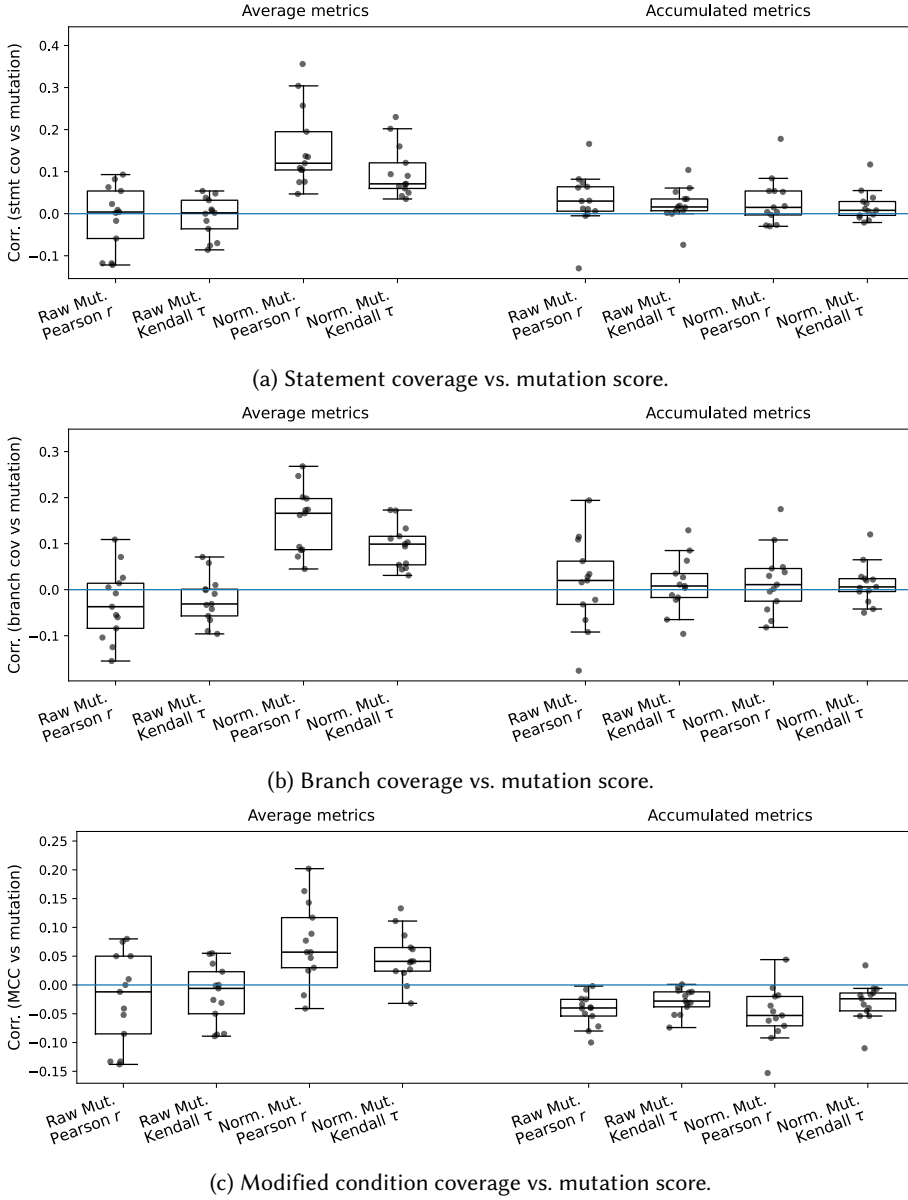


Fig. 3. Box plots of the correlation between coverage (statement, branch, and modified condition) and mutation score (raw and normalized) when test suite size is ignored. Each dot is the correlation computed from model-specific sampled test suite combinations; boxes group these correlations by aggregation type (average vs. accumulated) and statistic (Pearson r vs. Kendall τ).

Table 7 reports the inter-model results. Among coefficients that are statistically significant ($p < 0.05$), the correlations generally decrease relative to the size-unconstrained setting, but they are typically moderate and, in some cases, strong (e.g., branch coverage vs. raw mutation score at $k=10$, $r=0.780$). This pattern differs from Inozemtseva et al., who reported that once suite size is controlled, coverage correlates weakly with normalized mutation score and only moderately with

Table 7. Correlation between coverage and mutation score with test statistics aggregated into one data point for each model, for different fixed test suite sizes; p -values are reported in parentheses.

Suite size k	Coverage	Mutation score	Average aggregation		Accumulated aggregation	
			Pearson r	Kendall τ	Pearson r	Kendall τ
3	Statement	Raw	0.190 (0.534)	0.103 (0.675)	-0.144 (0.639)	-0.103 (0.675)
		Normalized	0.642 (0.018)	0.410 (0.057)	-0.251 (0.409)	-0.128 (0.590)
	Branch	Raw	0.525 (0.066)	0.282 (0.204)	-0.102 (0.739)	-0.026 (0.952)
		Normalized	0.579 (0.038)	0.487 (0.022)	0.036 (0.907)	0.000 (1.000)
	MCC	Raw	0.035 (0.910)	0.000 (1.000)	0.156 (0.610)	0.051 (0.858)
		Normalized	0.669 (0.012)	0.462 (0.030)	0.127 (0.680)	0.128 (0.590)
5	Statement	Raw	0.476 (0.100)	0.385 (0.076)	0.489 (0.090)	0.282 (0.204)
		Normalized	0.462 (0.112)	0.282 (0.204)	0.009 (0.977)	0.103 (0.675)
	Branch	Raw	0.699 (0.008)	0.513 (0.015)	-0.001 (0.997)	-0.154 (0.510)
		Normalized	0.323 (0.282)	0.256 (0.252)	0.165 (0.589)	0.179 (0.435)
	MCC	Raw	0.476 (0.100)	0.410 (0.057)	0.015 (0.962)	-0.077 (0.765)
		Normalized	0.526 (0.065)	0.410 (0.057)	-0.283 (0.350)	-0.154 (0.510)
10	Statement	Raw	0.632 (0.021)	0.590 (0.004)	0.140 (0.648)	0.179 (0.435)
		Normalized	0.688 (0.009)	0.513 (0.015)	-0.263 (0.386)	-0.282 (0.204)
	Branch	Raw	0.780 (0.002)	0.692 (0.001)	0.054 (0.860)	-0.154 (0.510)
		Normalized	0.528 (0.064)	0.410 (0.057)	0.072 (0.816)	-0.103 (0.675)
	MCC	Raw	0.550 (0.052)	0.462 (0.030)	0.095 (0.759)	0.077 (0.765)
		Normalized	0.652 (0.016)	0.487 (0.022)	-0.492 (0.088)	-0.333 (0.129)

raw mutation score. Overall, our results suggest that even after controlling suite size, coverage can still provide some indication of mutant-killing strength at the inter-model level.

4.4 Correlation Between Different Coverage Types

Beyond analyzing coverage versus mutation score, Inozemtseva et al. also examined correlations among different coverage metrics and found very strong relationships across coverage types. We replicate this analysis for LLM-generated test suites. Table 8 reports correlations among statement, branch, and modified condition coverage under both average and accumulated aggregation, for both unconstrained suites (All) and size-controlled suites.

Our results differ from those reported by Inozemtseva et al. in several respects. When suite size is unconstrained, coverage metrics are strongly correlated under average aggregation, but less so than the near-perfect relationships reported in 2014 (e.g., coefficients above 0.9 for both Pearson and Kendall τ between all coverage types). Pearson correlations typically remain strong, whereas Kendall τ is often only moderate. Under accumulated aggregation, statement and branch coverage remain strongly correlated, but correlations involving modified condition coverage often drop to moderate or weak. These gaps widen once suite size is controlled: under accumulated aggregation, modified condition coverage is sometimes only weakly correlated with statement and branch coverage, whereas under average aggregation the relationships remain mostly moderate to strong. Overall, these results do not support the claim from Inozemtseva et al. that different coverage types are essentially measuring the same thing in our LLM-generated setting.

4.5 RQ1 Summary

Overall, we observe several departures from the original findings:

- (1) **Test suite size vs. mutation score.** For LLM-generated tests, suite size is only weakly correlated with both raw and normalized mutation score, suggesting that generating more tests does not necessarily improve mutant-killing capability.
- (2) **Coverage vs. mutation score without size control.** Without controlling suite size, coverage-mutation correlations are weak in both the combined and intra-model views. In the inter-model view, however, coverage becomes moderately to strongly correlated with mutation score under average aggregation, indicating that coverage is informative for comparing models.

Table 8. Correlation between different coverage metrics (Statement, Branch, and Modified Condition) on sampled combinations of test suites, reported for the unconstrained setting (All) and for fixed suite sizes $k \in \{3, 5, 10\}$. p -values are reported in parentheses.

Suite size k	Coverage pair	Average aggregation		Accumulated aggregation	
		Pearson r	Kendall τ	Pearson r	Kendall τ
All	Statement vs Branch	0.753 (0.0)	0.553 (0.0)	0.857 (0.0)	0.675 (0.0)
	Statement vs MCC	0.885 (0.0)	0.694 (0.0)	0.319 (2.58×10^{-304})	0.222 (0.0)
	Branch vs MCC	0.709 (0.0)	0.510 (0.0)	0.400 (0.0)	0.271 (0.0)
3	Statement vs Branch	0.663 (0.0)	0.468 (0.0)	0.811 (0.0)	0.614 (0.0)
	Statement vs MCC	0.789 (0.0)	0.572 (0.0)	-0.003 (0.735)	0.005 (0.408)
	Branch vs MCC	0.458 (0.0)	0.303 (0.0)	-0.059 (1.99×10^{-11})	-0.037 (2.00×10^{-10})
5	Statement vs Branch	0.690 (0.0)	0.496 (0.0)	0.699 (0.0)	0.506 (0.0)
	Statement vs MCC	0.839 (0.0)	0.639 (0.0)	0.209 (1.15×10^{-128})	0.141 (3.99×10^{-129})
	Branch vs MCC	0.605 (0.0)	0.413 (0.0)	-0.021 (0.018)	-0.008 (0.190)
10	Statement vs Branch	0.715 (0.0)	0.517 (0.0)	0.772 (0.0)	0.566 (0.0)
	Statement vs MCC	0.878 (0.0)	0.683 (0.0)	0.189 (2.11×10^{-104})	0.130 (2.87×10^{-110})
	Branch vs MCC	0.665 (0.0)	0.467 (0.0)	0.034 (8.71×10^{-5})	0.027 (3.69×10^{-6})

- (3) **Coverage vs. mutation score with size control.** After fixing suite size, correlations remain weak in the combined and intra-model views; in the inter-model view they generally decrease but often remain moderate to strong, reinforcing that coverage is more informative for model comparison than for distinguishing suites within a model.
- (4) **Correlation among coverage metrics.** Correlations among coverage criteria are weaker than those reported by Inozemtseva et al.; under accumulated aggregation, modified condition coverage can be only weakly correlated with statement and branch coverage, suggesting that different coverage types are not interchangeable.

5 RQ2: Correlation Between Coverage and Real-Bug Detection

In RQ2, we move beyond using mutation score as a proxy, as done by Inozemtseva et al., and analyze the correlation between coverage and real-bug detection ratio as the effectiveness measure.

5.1 Is Test Suite Size Correlated With Real-Bug Detection?

Unless otherwise noted, the analyses in this and the following two subsections use test suites generated from the bug-free (fixed) versions of the focal methods; the buggy-input setting is examined in Section 5.4. As in RQ1, we begin by examining test suite size. The overall trend is consistent in both the combined and intra-model views, where correlation coefficients remain weak. In the inter-model view, the correlation increases slightly to around 0.4, but stays just below the moderate threshold and is not statistically significant, likely due to the small sample size. Overall, these results suggest that test suite size is not strongly correlated with real-bug detection ratio for LLM-generated tests, mirroring the weak relationship we observed for mutation score.

Table 9. Correlation between coverage and bug detection ratio across combinations of randomly selected test suites from all models; all entries are significant ($p < 0.05$).

Coverage metric	Average aggregation		Accumulated aggregation	
	Pearson r	Kendall τ	Pearson r	Kendall τ
Statement	0.368	0.250	0.473	0.336
Branch	0.385	0.266	0.481	0.337
MCC	0.379	0.261	0.190	0.127

5.2 Is Coverage Correlated With Real-Bug Detection When Test Size Is Ignored?

The overall pattern for coverage versus real-bug detection mirrors what we observed for coverage versus mutation score. In the combined view (Table 9), coverage exhibits a weak-to-moderate correlation with bug detection effectiveness: all coefficients stay below 0.5, with accumulated modified condition coverage as the clearest outlier ($r=0.190$). As with the coverage-mutation analysis, switching to the intra-model view yields much weaker correlations (typically below 0.2). This drop suggests that the stronger combined-view correlation might be driven primarily by differences between models, motivating a closer look at the inter-model view.

Table 10. Inter-model correlation between coverage and bug detection ratio, with statistics aggregated into one data point per model; p -values are reported in parentheses.

Coverage metric	Average aggregation		Accumulated aggregation	
	Pearson r	Kendall τ	Pearson r	Kendall τ
Statement	0.617 (0.025)	0.477 (0.024)	0.619 (0.024)	0.555 (0.0086)
Branch	0.861 (1.6×10^{-4})	0.761 (3.1×10^{-4})	0.542 (0.056)	0.400 (0.058)
MCC	0.573 (0.041)	0.503 (0.017)	0.504 (0.079)	0.452 (0.032)

We report the inter-model results in Table 10. Overall, the correlations between coverage and bug detection ratio are moderate to strong, and the strongest correlations are also the most statistically significant. In particular, average branch coverage shows a strong relationship with bug detection ratio and yields the smallest p -values. Unlike the mutation-score results, accumulated coverage metrics also maintain moderate correlations with bug detection ratio; among them, statement coverage stands out as both statistically significant and relatively strong. Overall, these results suggest that coverage can be a useful indicator of bug detection effectiveness when comparing across different models.

Table 11. Inter-model correlation between coverage and bug detection ratio, with statistics aggregated into one data point per model, reported for different fixed test suite sizes k . p -values are reported in parentheses.

k	Coverage metric	Average aggregation		Accumulated aggregation	
		Pearson r	Kendall τ	Pearson r	Kendall τ
3	Statement	0.205 (0.501)	0.222 (0.297)	0.384 (0.195)	0.118 (0.581)
	Branch	0.679 (0.011)	0.405 (0.057)	0.351 (0.240)	0.248 (0.244)
	MCC	0.098 (0.751)	0.118 (0.581)	-0.175 (0.568)	-0.118 (0.581)
5	Statement	0.528 (0.064)	0.536 (0.012)	0.690 (0.0091)	0.484 (0.023)
	Branch	0.855 (2.0×10^{-4})	0.667 (0.0017)	0.183 (0.549)	-0.013 (0.951)
	MCC	0.554 (0.050)	0.510 (0.017)	0.104 (0.735)	0.144 (0.499)
10	Statement	0.589 (0.034)	0.529 (0.012)	0.496 (0.085)	0.297 (0.160)
	Branch	0.858 (1.7×10^{-4})	0.735 (4.9×10^{-4})	0.353 (0.237)	0.168 (0.427)
	MCC	0.587 (0.035)	0.555 (0.0086)	0.121 (0.694)	0.142 (0.501)

5.3 Is Coverage Correlated With Real-Bug Detection When Test Size Is Fixed?

As in the size-unconstrained setting, the intra-model correlations remain weak; the combined-view correlations are consistently stronger but fall below the moderate threshold once suite size is controlled. Since these trends mirror what we observed for mutation score, we omit their detailed discussion here and focus on the inter-model analysis. Table 11 reports the model-wise correlations for each fixed suite size. Among the statistically significant results, correlations are consistently moderate to strong, with branch coverage standing out in particular: it is strong for $k=5$ and $k=10$,

and remains just below the “strong” threshold for $k=3$ ($r=0.679$). Under accumulated aggregation, correlations generally drop relative to the unconstrained setting, with the only statistically significant correlation appearing between statement coverage and bug detection for $k=5$.

Overall, the fixed-size inter-model results reinforce our earlier finding that coverage can be a meaningful indicator of bug detection effectiveness when comparing models. The signal is strongest for average branch coverage, suggesting that different coverage types can indeed convey different information about the practical effectiveness of LLM-generated tests.

Bug-free focal method	Buggy focal method
<pre> public static boolean equals(CharSequence cs1, CharSequence cs2) { if (cs1 == cs2) return true; if (cs1 == null cs2 == null) return false; if (cs1 instanceof String && cs2 instanceof String) return cs1.equals(cs2); return CharSequenceUtils.regionMatches(cs1, false, 0, cs2, 0, Math.max(cs1.length(), cs2.length())); } </pre>	<pre> public static boolean equals(CharSequence cs1, CharSequence cs2) { if (cs1 == cs2) return true; if (cs1 == null cs2 == null) return false; return cs1.equals(cs2); } </pre>
⇓ test generated by the LLM ⇓	
Asserts the intended behavior <pre> @Test public void testEquals_stringBuilder() { assertTrue(StringUtils.equals("abc", new StringBuilder("abc"))); } </pre>	Asserts the buggy behavior <pre> @Test public void testEquals_stringBuilder() { assertFalse(StringUtils.equals("abc", new StringBuilder("abc"))); } </pre>

Fig. 4. Representative example comparing tests generated from the bug-free (left) and buggy (right) implementations of `StringUtils.equals` (Defects4J Lang-14) using the same LLM and prompt template. The top row shows the focal-method bodies provided to the model, and the bottom row shows the corresponding generated test methods. Highlighted text marks the key differences between the two implementations and between the two generated tests.

5.4 Does the Strong Correlation We Observe Still Hold When the Input Code Is Buggy?

Our inter-model analyses so far show that coverage can correlate moderately, and for some metrics even strongly, with bug detection effectiveness—but this evidence is obtained when tests are generated from bug-free code. In practice, however, the correctness of the code-under-test is not guaranteed, and the goal of testing is often to expose bugs in the buggy code itself. Prior work by Huang et al. [29] shows that prompting LLMs with buggy code can still produce tests that expose the bug, but the number of such bug-detecting tests is substantially lower than when tests are generated from the corresponding bug-free code. A major reason is that LLMs can be misled by the buggy implementation and generate tests that assert the erroneous behavior of the buggy code rather than the intended behavior.

Figure 4 illustrates one such case using `StringUtils.equals` from Lang-14 in Defects4J. The method is intended to compare the contents of two `CharSequence` objects, where `CharSequence` is a Java interface implemented by common text types such as `String`, `StringBuilder`, and `StringBuffer`. Thus, “abc” and `new StringBuilder(“abc”)` should be considered equal despite having different concrete types. The bug-free implementation handles this case through `regionMatches`, whereas the buggy implementation directly calls `cs1.equals(cs2)`. Since `String.equals` returns true only when its argument is also a `String` with the same contents, it returns false when comparing a `String` against a `StringBuilder`. When prompted with this buggy implementation, the LLM treats this faulty behavior as intended and generates a test that asserts the

incorrect outcome via `assertFalse`; in contrast, the test generated from the bug-free implementation asserts the intended behavior with `assertTrue`. This example illustrates how tests generated from buggy code can encode faulty behavior that is absent from the bug-free implementation.

Still, it remains unclear whether higher coverage on buggy code is helpful: even if an LLM treats faulty behavior as intended and thus produces weaker or incorrect assertions, exercising a broader portion of the implementation could, in principle, still increase the chance of triggering and exposing buggy behavior.

To examine this, we repeat the same correlation-analysis procedure used for bug-free code, but instead generate tests from buggy inputs and correlate coverage measured on the buggy version with bug detection effectiveness. Due to space constraints, and because the trend is consistent for all settings, we do not report the full set of coefficients here; complete statistics are available in our replication package. In summary, the correlations are weak under the combined, intra-model, and inter-model views. Although this outcome is not entirely surprising—coverage on buggy code is not necessarily informative about whether tests will detect that bug—the contrast with the bug-free setting is practically important, as it highlights a key limitation of recent LLM-based testing evaluation and test-generation practices that predominantly assume bug-free code as input.

5.5 RQ2 Summary

Overall, for test suites generated from bug-free code, coverage correlates more strongly with real-bug detection than with mutation score in RQ1: when comparing *across* models, most coverage metrics are moderately correlated with bug detection effectiveness, and some criteria exhibit strong correlations. However, when the code-under-test is buggy, correlations between coverage and bug detection become uniformly weak across all three views, indicating that coverage measured on the buggy code is not informative of whether the generated tests detect that bug.

6 RQ3: Correlation Between Mutation and Real-Bug Detection

For RQ3, we leverage the data from RQ1 and RQ2 to replicate Papadakis et al.’s analysis of the relationship between mutation score and real-bug detection effectiveness for LLM-generated tests.

6.1 Is Mutation Score Correlated With Real-Bug Detection When Test Size Is Ignored?

We begin with the combined view (Table 12). The Pearson correlation between mutation score and bug detection ratio is moderate for average aggregation (0.475–0.511), while the corresponding Kendall τ stays below the moderate threshold; correlations are weak for accumulated aggregation. In the intra-model view, correlations drop to weak across all models and metric types (detailed results omitted for brevity). This pattern mirrors RQ1 and RQ2, suggesting that any stronger relationship, if present, is more likely to appear at the inter-model level.

Table 12. Correlation between mutation and bug detection ratio across combinations of randomly selected test suites from all models, reported for both average aggregation and accumulated aggregation; all entries are significant ($p < 0.05$).

Mutation score	Average aggregation		Accumulated aggregation	
	Pearson r	Kendall τ	Pearson r	Kendall τ
Raw	0.475	0.320	0.146	0.110
Normalized	0.511	0.373	0.196	0.123

Table 13 reports the inter-model results. Under both aggregation schemes, raw mutation score shows the strongest association with bug detection effectiveness: the correlation is strong under

average aggregation and moderate under accumulated aggregation. This suggests that raw mutation score can serve as a useful indicator of bug detection effectiveness when comparing models. Notably, the metric variant with the strongest correlation differs from that in the combined and intra-model views. This pattern contrasts with earlier RQs, where the relative ordering of metric variants was more consistent across views, reinforcing the importance of reporting multiple metric variants.

Table 13. Inter-model correlation between mutation and bug detection ratio, with statistics aggregated into one data point per model; p -values are reported in parentheses.

Mutation score	Average aggregation		Accumulated aggregation	
	Pearson r	Kendall τ	Pearson r	Kendall τ
Raw	0.863 (1.44×10^{-4})	0.761 (3.11×10^{-4})	0.622 (0.023)	0.529 (0.012)
Normalized	0.493 (0.087)	0.374 (0.076)	-0.077 (0.803)	-0.039 (0.855)

6.2 Is Mutation Score Correlated With Real-Bug Detection When Test Size Is Fixed?

In the combined and intra-model views, the size-controlled results largely follow the same overall trend as in the size-unconstrained setting; therefore, we omit detailed coefficients for brevity and summarize the results here. In both views, correlations remain weak, with intra-model correlations consistently weaker than the combined results and average aggregation consistently stronger than accumulated aggregation. In the inter-model view (Table 14), correlations decrease slightly after controlling for suite size but mostly remain moderate to strong and statistically significant for raw mutation score under both average and accumulated aggregation, reinforcing our observations from the previous section.

Table 14. Inter-model correlation between mutation and bug detection ratio at fixed test suite size k , with statistics aggregated into one data point per model; p -values are reported in parentheses.

Suite size k	Mutation score	Average aggregation		Accumulated aggregation	
		Pearson r	Kendall τ	Pearson r	Kendall τ
3	Raw	0.702 (0.007)	0.510 (0.017)	0.585 (0.036)	0.405 (0.057)
	Normalized	0.354 (0.236)	0.222 (0.297)	0.186 (0.542)	0.013 (0.951)
5	Raw	0.695 (0.008)	0.510 (0.017)	0.533 (0.061)	0.301 (0.158)
	Normalized	0.386 (0.192)	0.275 (0.197)	-0.186 (0.542)	-0.118 (0.580)
10	Raw	0.833 (4.05×10^{-4})	0.658 (1.83×10^{-3})	0.600 (0.030)	0.477 (0.024)
	Normalized	0.532 (0.061)	0.426 (0.044)	-0.085 (0.783)	0.013 (0.951)

6.3 RQ3 Summary

For RQ3, we again observe a departure from the original findings of Papadakis et al. For LLM-generated tests, the correlation between mutation score and real-bug detection ratio remains at most moderate in the combined view and weak in the intra-model view, regardless of whether test suite size is controlled. In contrast, in the inter-model view, bug detection is moderately to strongly correlated with *raw* mutation score under both average and accumulated aggregation, suggesting that raw mutation score can be informative for comparing bug detection effectiveness across models.

We also observe a different pattern from our earlier results: whereas differences were previously more often attributable to the choice of aggregation (average vs. accumulated) than to the mutation-score variant itself, in RQ3 the mutation-score variant plays a larger role. Overall, these findings motivate reporting multiple metrics under both aggregation schemes.

7 Discussion

Based on our findings, we summarize a set of implications, insights, and actionable recommendations that we believe can inform better practice for evaluating LLMs for test generation and for interpreting evaluation results.

① **Coverage and mutation can be informative, but only under certain conditions.** In contrast to the conclusions of Inozemtseva et al. and Papadakis et al., we find that coverage and mutation can still be indicative of bug detection effectiveness, even when test suite size is controlled. However, this holds only in certain settings. When the code provided to the LLM can reasonably be assumed bug-free (e.g., regression-style scenarios where the goal is to detect bugs introduced by future changes), higher coverage and mutation scores often coincide with higher real-bug detection effectiveness when comparing *across* models. In the more practical setting where the code-under-test may already be buggy and the goal is to expose that existing bug, coverage loses predictive power for bug detection effectiveness, and mutation testing is not applicable.

② **Evaluating LLMs under the practical scenario of buggy code-under-test.** Given the weak correlations we observe between coverage and bug detection effectiveness in the buggy-code setting—together with the limited applicability of mutation analysis—we emphasize the need to directly evaluate an LLM’s ability to generate tests that detect bugs in the code-under-test. This conclusion aligns with Huang et al. [29], who show that models’ bug detection effectiveness drops substantially when the input code is buggy. Taken together, these findings motivate further research on methods that improve LLMs’ ability to generate tests that expose existing bugs.

③ **Different coverage criteria provide complementary signals.** Inozemtseva et al. reported very strong correlations among different coverage metrics and concluded that “Additionally, it may be in the developer’s best interest to use simpler coverage measures. These measures provide a similar amount of information about the suite’s effectiveness but introduce less measurement overhead.” However, we find that, for LLM-generated tests, this conclusion does not hold. In our results, correlations between coverage criteria are only moderate to strong depending on the particular pair, and this pattern is primarily observed under the *average* aggregation. Under the *accumulated* aggregation, modified condition coverage correlates only weakly with statement and branch coverage. These observations suggest that reporting multiple coverage criteria, including stricter ones such as modified condition coverage, can provide additional information about LLM-generated test suites. This is also consistent with industrial practice in safety-critical domains, where standards such as DO-178B require adequate modified condition/decision coverage [25].

④ **Is test suite size really a confounding factor?** While both Inozemtseva et al. and Papadakis et al. argue that test suite size is a strong confounder in the relationships among coverage, mutation, and real-bug detection, our results for LLM-generated tests provide little support for this conclusion. We find that test suite size correlates only weakly with mutation score and with real-bug detection ratio, with at most a modest increase toward the moderate threshold in the inter-model view for bug detection. We further examine correlations between test suite size and coverage and again observe no strong relationship. Moreover, moving from the size-unconstrained to the fixed-size setting yields only a small drop in correlations: they largely retain their qualitative strength, with only occasional drops from strong to moderate. Taken together, these results suggest that test suite size is unlikely to be the primary driver of the strong correlations we observe.

⑤ **Why do our conclusions about test suite size differ substantially from prior work?** The weaker confounding effect of test suite size in our study likely stems from how varying-sized test suites are constructed. In our setting, each test suite is generated by the LLM for a single focal method, so differences in suite size likely reflect how many tests are needed to exercise the set of behaviors required to validate that method’s expected functionality. In this sense, variation in suite

size arises naturally from the testing needs of the focal method, rather than from an externally imposed target size, and therefore is less likely to exert a strong confounding effect on the other metrics. In contrast, prior studies construct suites by sampling a specified number of tests from a project-wide test pool. Under that setup, each sampled test contributes only part of the overall behavior space, so larger sampled suites mechanically aggregate more of these partial contributions and tend to achieve higher coverage, mutant killing, and bug detection simply because they include more tests, creating a stronger size-driven confounding effect.

⑥ How should we interpret test suite size-related metrics for LLM-generated test suites?

Our discussion in ④ and ⑤ also informs how to interpret test-correctness measures (e.g., the number or fraction of generated tests that compile or pass), which are commonly reported in LLM test-generation evaluations [45, 57, 58]. While producing more “correct” tests is desirable, correctness measures should not be conflated with bug detection effectiveness; instead, they are better viewed as measures of *cost-effectiveness*. In particular, a higher fraction of compilable/passing tests indicates that less generation budget is wasted on unusable outputs.

⑦ **What other factors may contribute to the discrepancies between our findings and prior studies?** Beyond the difference in test suite construction discussed in ⑤, we identify two further differences that may help account for why our findings diverge from prior studies. First, LLM-based test generation differs from both human test writing and traditional automated test generation because both the behaviors to be exercised and the assertion oracles are conditioned on the prompt, focal method, and surrounding code context. By contrast, human-written tests are often guided by developers’ understanding of the intended behavior, while traditional automated tools typically take the implementation as input and generate regression tests by optimizing structural testing objectives, such as coverage or mutation, over that implementation. Second, we examine correlations at multiple granularities, including combined, intra-model, and inter-model analyses, revealing that correlations can vary substantially across different perspectives, an analysis not conducted in the same way by prior studies.

8 Threats to Validity

External Validity. External validity concerns the generalizability of our findings beyond the studied setting. Following the scope of Inozemtseva et al., we focus on large open-source Java projects and use Defects4J, a widely adopted benchmark in both prior automated testing research and recent evaluations of LLM-based unit test generation. To reduce the risk that our results are driven by a small sample or a particular model, we conduct experiments across 11 state-of-the-art LLMs and analyze more than 100,000 distinct generated test cases. Nevertheless, our conclusions are limited to the Java ecosystem and the characteristics of Defects4J projects, and may not fully generalize to other programming languages or project styles. That said, the scale of our experiments should be sufficient to support the key differences we observe relative to prior studies. A second threat is that our conclusions are based on a single prompting workflow. Since identifying a universally “best” prompting strategy is unrealistic, we adopt a prompt template that is well justified by prior work and can be fully automated using only the code-under-test, without additional human effort, aligning with practical test-generation scenarios. We acknowledge, however, that alternative prompt designs or different contexts may yield different relationships among the studied metrics.

Internal Validity. A threat to internal validity is that some of our conclusions rely on the inter-model view, which involves a relatively small number of data points (one per model) and can reduce statistical power while increasing sensitivity to outliers. To mitigate this threat, we interpret inter-model results primarily through statistically significant coefficients and check whether the observed trends are consistent across multiple complementary settings, including randomly sampled test

suites with both uncontrolled and controlled size. The fact that the same patterns persist across these settings suggests that our main observations are unlikely to be driven purely by chance.

Construct Validity. A threat to construct validity is the inherent stochasticity of LLM outputs and the dependence of results on prompting and metric choices. To reduce output variability, we set the generation temperature to 0 to maximize determinism, while acknowledging that some nondeterminism may still arise from implementation details. Another potential threat is data contamination: developer-written tests in Defects4J may have appeared in model training data, which could inflate measured performance. While we cannot rule this out, we observe substantial differences between tests generated from buggy versus fixed versions of the same focal methods (which share the same Defects4J oracle tests), and these differences are reflected in our measured relationships among metrics; this suggests that models are responding to the provided prompt context rather than simply reproducing memorized tests.

9 Conclusion and Future Work

In this paper, we conduct a large-scale replication study of Inozemtseva et al. and Papadakis et al. to examine how widely used proxy metrics—test coverage and mutation score—relate to the real-bug detection effectiveness of LLM-generated test suites. Our conclusions differ substantially from those of the two prior studies. In particular, we find that the usefulness of coverage and mutation is highly context-dependent. When the code provided to the LLM can be reasonably assumed bug-free, certain forms of coverage and mutation provide meaningful signals when comparing *across* models. In contrast, in the practically common scenario where the code-under-test may already be buggy and the goal is to expose that bug, coverage becomes unreliable and mutation analysis is not applicable. We also find little evidence that test suite size is a dominant confounder for correlations among coverage, mutation, and real-bug detection for LLM-generated tests. Taken together, these results help interpret prior findings and motivate future evaluations to measure what matters most: whether generated tests can reveal bugs in the code-under-test.

Our study also suggests several directions for future work. First, prior LLM-based test generation studies adopt a wide range of prompt templates and prompting strategies; a natural next step is to systematically vary prompt construction (e.g., the amount and type of code context, formatting, and auxiliary instructions) and examine how these choices affect the observed relationships among coverage, mutation, and real-bug detection effectiveness. Second, many recent techniques improve proxy outcomes such as compilation success, pass rate, coverage, or mutation score through specialized prompting or post-processing; it remains unclear when such improvements translate into higher *real-bug detection effectiveness*, and clarifying this linkage would help determine when optimizing these proxies is justified. Third, our experiments adopt a predominantly white-box prompting setting in which the code-under-test is provided; future work should assess whether our observations hold in black-box or specification-based settings, where models generate tests from natural-language specifications, docstrings, or other behavioral descriptions without access to the implementation. Fourth, our study focuses on the overall correlations among coverage, mutation, and real-bug detection effectiveness for LLM-generated Java tests, but these relationships may vary across projects; investigating such project-specific correlations is therefore an important direction for future work. Finally, future work should compare tests from different sources—LLM-generated, human-written, and non-LLM test automation tools—to understand how these relationships differ across settings and what drives those differences.

Acknowledgments

This work was supported by the Natural Sciences and Engineering Research Council of Canada (NSERC) under Grant RGPIN-2022-04154.

Data Availability

The data used in this study is drawn from the publicly available Defects4J dataset [34]. To support reproducibility, we provide a replication package [61] containing our test-generation code, data-preprocessing scripts, and the raw data used in our analysis. All results reported in the paper can be reproduced using this package. The package is publicly available on GitHub at https://github.com/drixxs2050/Cov_mut_bug_detect_correlation and permanently archived on Zenodo (DOI: 10.5281/zenodo.21429528).

References

- [1] 2025. CodeCover. <http://codecover.org/>. Accessed: 2025-09-10.
- [2] Azat Abdullin, Pouria Derakhshanfar, and Annibale Panichella. 2025. Test Wars: A Comparative Study of SBST, Symbolic Execution, and LLM-Based Approaches to Unit Test Generation. arXiv:2501.10200 [cs.SE] doi:10.48550/arXiv.2501.10200
- [3] Saranya Alagarsamy, Chakkrit Tantithamthavorn, and Aldeida Aleti. 2024. A3Test: Assertion-Augmented Automated Test Case Generation. *Information and Software Technology* 176 (2024), 107565. doi:10.1016/j.infsof.2024.107565
- [4] J. H. Andrews, L. C. Briand, and Y. Labiche. 2005. Is mutation an appropriate tool for testing experiments?. In *Proceedings of the 27th International Conference on Software Engineering* (St. Louis, MO, USA) (ICSE '05). Association for Computing Machinery, New York, NY, USA, 402–411. doi:10.1145/1062455.1062530
- [5] Anthropic. 2025. Claude 4 Sonnet. <https://www.anthropic.com/claude/sonnet>. Accessed: 2025-05-23.
- [6] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. 2021. Program Synthesis with Large Language Models. arXiv:2108.07732 [cs.PL] doi:10.48550/arXiv.2108.07732
- [7] Shreya Bhatia, Tarushi Gandhi, Dhruv Kumar, and Pankaj Jalote. 2024. Unit Test Generation using Generative AI: A Comparative Performance Analysis of Autogeneration Tools. In *Proceedings of the 1st International Workshop on Large Language Models for Code* (Lisbon, Portugal) (LLM4Code '24). Association for Computing Machinery, New York, NY, USA, 54–61. doi:10.1145/3643795.3648396
- [8] Max Brunsfeld. 2018. Tree-sitter: An incremental parsing system for programming tools. Accessed: 2025-02-21. doi:10.5281/zenodo.4619183
- [9] Xia Cai and Michael R. Lyu. 2005. The effect of code coverage on fault detection under different testing profiles. *SIGSOFT Softw. Eng. Notes* 30, 4 (July 2005), 1–7. doi:10.1145/1082983.1083288
- [10] Jeffrey C. Carver, Natalia Juristo, Maria Teresa Baldassarre, and Sira Vegas. 2014. Replications of software engineering experiments. *Empirical Software Engineering* 19, 2 (2014), 267–276. doi:10.1007/s10664-013-9290-8
- [11] Mark Chen et al. 2021. Evaluating Large Language Models Trained on Code. arXiv:2107.03374 [cs.LG] doi:10.48550/arXiv.2107.03374
- [12] Yinghao Chen, Zehao Hu, Chen Zhi, Junxiao Han, Shuiguang Deng, and Jianwei Yin. 2024. ChatUniTest: A Framework for LLM-Based Test Generation. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering* (Porto de Galinhas, Brazil) (FSE 2024). Association for Computing Machinery, New York, NY, USA, 572–576. doi:10.1145/3663529.3663801
- [13] Henry Coles, Thomas Laurent, Christopher Henard, Mike Papadakis, and Anthony Ventresque. 2016. PIT: a practical mutation testing tool for Java (demo). In *Proceedings of the 25th International Symposium on Software Testing and Analysis* (Saarbrücken, Germany) (ISSTA 2016). Association for Computing Machinery, New York, NY, USA, 449–452. doi:10.1145/2931037.2948707
- [14] Arghavan Moradi Dakhel, Amin Nikanjam, Vahid Majdinasab, Foutse Khomh, and Michel C. Desmarais. 2024. Effective test generation using pre-trained Large Language Models and mutation testing. *Information and Software Technology* 171 (2024), 107468. doi:10.1016/j.infsof.2024.107468
- [15] DeepSeek-AI. 2025. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. arXiv:2501.12948 [cs.CL] doi:10.48550/arXiv.2501.12948
- [16] DeepSeek-AI. 2025. DeepSeek-V3 Technical Report. arXiv:2412.19437 [cs.CL] doi:10.48550/arXiv.2412.19437
- [17] Phyllis G. Frankl and Oleg Iakounenko. 1998. Further empirical studies of test effectiveness. In *Proceedings of the 6th ACM SIGSOFT International Symposium on Foundations of Software Engineering* (Lake Buena Vista, Florida, USA) (SIGSOFT '98/FSE-6). Association for Computing Machinery, New York, NY, USA, 153–162. doi:10.1145/288195.288298
- [18] Gordon Fraser and Andrea Arcuri. 2014. A Large-Scale Evaluation of Automated Unit Test Generation Using EvoSuite. *ACM Trans. Softw. Eng. Methodol.* 24, 2, Article 8 (Dec. 2014), 42 pages. doi:10.1145/2685612
- [19] Gordon Fraser, Matt Staats, Phil McMinn, Andrea Arcuri, and Frank Padberg. 2013. Does automated white-box test generation really help software testers?. In *Proceedings of the 2013 International Symposium on Software Testing and Analysis* (Lugano, Switzerland) (ISSTA 2013). Association for Computing Machinery, New York, NY, USA, 291–301.

- doi:10.1145/2483760.2483774
- [20] Milos Gligoric, Alex Groce, Chaoqiang Zhang, Rohan Sharma, Mohammad Amin Alipour, and Darko Marinov. 2013. Comparing non-adequate test suites using coverage criteria. In *Proceedings of the 2013 International Symposium on Software Testing and Analysis* (Lugano, Switzerland) (ISSTA 2013). Association for Computing Machinery, New York, NY, USA, 302–313. doi:10.1145/2483760.2483769
 - [21] Google. 2025. Gemini 2.5 Flash Model. <https://ai.google.dev/gemini-api/docs/models#gemini-2.5-flash>. Accessed: 2025-08-19.
 - [22] Google. 2025. Gemini 2.5 Pro Model. <https://ai.google.dev/gemini-api/docs/models#gemini-2.5-pro>. Accessed: 2025-08-19.
 - [23] J.P. Guilford. 1956. *Fundamental Statistics in Psychology and Education*. McGraw-Hill. <https://books.google.ca/books?id=u-G10ZqLhtsC>
 - [24] Vitor Guilherme and Auri Vincenzi. 2023. An initial investigation of ChatGPT unit test generation capability. In *Proceedings of the 8th Brazilian Symposium on Systematic and Automated Software Testing* (Campo Grande, MS, Brazil) (SAST '23). Association for Computing Machinery, New York, NY, USA, 15–24. doi:10.1145/3624032.3624035
 - [25] Kelly J. Hayhurst and Dan S. Veerhusen. 2001. A practical approach to modified condition/decision coverage. In *Proceedings of the 20th Digital Avionics Systems Conference (DASC)*, Vol. 1. IEEE, 1B2/1–1B2/10. doi:10.1109/DASC.2001.963305
 - [26] Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Akul Arora, Ethan Guo, Collin Burns, Samir Puranik, Horace He, Dawn Song, and Jacob Steinhardt. 2021. Measuring Coding Challenge Competence With APPS. In *Advances in Neural Information Processing Systems (Datasets and Benchmarks Track)*. doi:10.48550/arXiv.2105.09938
 - [27] Soneya Binta Hossain, Raygan Taylor, and Matthew Dwyer. 2025. Doc2OracLL: Investigating the Impact of Documentation on LLM-Based Test Oracle Generation. *Proc. ACM Softw. Eng.* 2, FSE, Article FSE084 (June 2025), 22 pages. doi:10.1145/3729354
 - [28] Xing Hu, Zhuang Liu, Xin Xia, Zhongxin Liu, Tongtong Xu, and Xiaohu Yang. 2023. Identify and Update Test Cases When Production Code Changes: A Transformer-Based Approach. In *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering* (Echternach, Luxembourg) (ASE '23). IEEE Press, 1111–1122. doi:10.1109/ASE56229.2023.00165
 - [29] Dong Huang, Jie M. Zhang, Mark Harman, Mingzhe Du, and Heming Cui. 2025. Measuring the Influence of Incorrect Code on Test Generation. arXiv:2409.09464 [cs.SE] doi:10.48550/arXiv.2409.09464
 - [30] Laura Inozemtseva and Reid Holmes. 2014. Coverage is not strongly correlated with test suite effectiveness. In *Proceedings of the 36th International Conference on Software Engineering* (Hyderabad, India) (ICSE 2014). Association for Computing Machinery, New York, NY, USA, 435–445. doi:10.1145/2568225.2568271
 - [31] Kush Jain, Gabriel Synnaeve, and Baptiste Roziere. 2025. TestGenEval: A Real World Unit Test Generation and Test Completion Benchmark. In *The Thirteenth International Conference on Learning Representations*. doi:10.48550/arXiv.2410.00752
 - [32] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues?. In *The Twelfth International Conference on Learning Representations*. doi:10.48550/arXiv.2310.06770
 - [33] Paul C. Jorgensen. 2013. *Software Testing: A Craftsman's Approach* (fourth ed.). Auerbach Publications. doi:10.1201/b15980
 - [34] René Just, Darioush Jalali, and Michael D. Ernst. 2014. Defects4J: a database of existing faults to enable controlled testing studies for Java programs. In *Proceedings of the 2014 International Symposium on Software Testing and Analysis* (San Jose, CA, USA) (ISSTA 2014). Association for Computing Machinery, New York, NY, USA, 437–440. doi:10.1145/2610384.2628055
 - [35] René Just, Darioush Jalali, Laura Inozemtseva, Michael D. Ernst, Reid Holmes, and Gordon Fraser. 2014. Are mutants a valid substitute for real faults in software testing?. In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering* (Hong Kong, China) (FSE 2014). Association for Computing Machinery, New York, NY, USA, 654–665. doi:10.1145/2635868.2635929
 - [36] Andrea Lops, Fedelucio Narducci, Azzurra Ragone, Michelantonio Trizio, and Claudio Bartolini. 2025. A System for Automated Unit Test Generation using Large Language Models and Assessment of Generated Test Suites. In *2025 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW)*. IEEE, 29–36. doi:10.1109/icstw64639.2025.10962454
 - [37] OpenAI. 2025. GPT-4.1. <https://openai.com/index/gpt-4-1/>. Accessed: 2025-08-19.
 - [38] OpenAI. 2025. OpenAI Models - O4 Mini. <https://platform.openai.com/docs/models/o4-mini>. Accessed: 2025-08-19.
 - [39] Annibale Panichella, Fitsum Meshesha Kifetew, and Paolo Tonella. 2018. Automated Test Case Generation as a Many-Objective Optimisation Problem with Dynamic Selection of the Targets. *IEEE Transactions on Software Engineering* 44, 2 (2018), 122–158. doi:10.1109/TSE.2017.2663435

- [40] Mike Papadakis, Donghwan Shin, Shin Yoo, and Doo-Hwan Bae. 2018. Are mutation scores correlated with real fault detection? A large scale empirical study on the relationship between mutants and real faults. In *Proceedings of the 40th International Conference on Software Engineering* (Gothenburg, Sweden) (ICSE '18). Association for Computing Machinery, New York, NY, USA, 537–548. doi:10.1145/3180155.3180183
- [41] Qwen Team. 2025. Qwen3-Coder: Agentic Coding in the World. <https://qwenlm.github.io/blog/qwen3-coder/>. Accessed: 2025-08-19.
- [42] Qwen Team. 2025. Qwen3: Think Deeper, Act Faster. <https://qwenlm.github.io/blog/qwen3/>. Accessed: 2025-08-19.
- [43] Nikitha Rao, Kush Jain, Uri Alon, Claire Le Goues, and Vincent J. Hellendoorn. 2023. CAT-LM Training Language Models on Aligned Code and Tests. In *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering* (Echternach, Luxembourg) (ASE '23). IEEE Press, 409–420. doi:10.1109/ASE56229.2023.00193
- [44] José Miguel Rojas, Gordon Fraser, and Andrea Arcuri. 2015. Automated unit test generation during software development: a controlled experiment and think-aloud observations. In *Proceedings of the 2015 International Symposium on Software Testing and Analysis* (Baltimore, MD, USA) (ISSTA 2015). Association for Computing Machinery, New York, NY, USA, 338–349. doi:10.1145/2771783.2771801
- [45] Max Schäfer, Sarah Nadi, Aryaz Eghbali, and Frank Tip. 2024. An Empirical Evaluation of Using Large Language Models for Automated Unit Test Generation. *IEEE Transactions on Software Engineering* 50, 1 (2024), 85–105. doi:10.1109/TSE.2023.3334955
- [46] Ye Shang, Quanjun Zhang, Chunrong Fang, Siqi Gu, Jianyi Zhou, and Zhenyu Chen. 2025. A Large-Scale Empirical Study on Fine-Tuning Large Language Models for Unit Testing. *Proc. ACM Softw. Eng.* 2, ISSTA, Article ISSTA074 (June 2025), 23 pages. doi:10.1145/3728951
- [47] Forrest J. Shull, Jeffrey C. Carver, Sira Vegas, and Natalia Juristo. 2008. The role of replications in Empirical Software Engineering. *Empirical Softw. Engg.* 13, 2 (April 2008), 211–218. doi:10.1007/s10664-008-9060-1
- [48] Mohammed Latif Siddiq, Joanna Cecilia Da Silva Santos, Ridwanul Hasan Tanvir, Noshin Ulfat, Fahmid Al Rifat, and Vinicius Carvalho Lopes. 2024. Using Large Language Models to Generate JUnit Tests: An Empirical Study. In *Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering* (Salerno, Italy) (EASE '24). Association for Computing Machinery, New York, NY, USA, 313–322. doi:10.1145/3661167.3661216
- [49] André Silva, Nuno Saavedra, and Martin Monperrus. 2024. GitBug-Java: A Reproducible Benchmark of Recent Java Bugs. In *2024 IEEE/ACM 21st International Conference on Mining Software Repositories (MSR)*. IEEE Computer Society, Los Alamitos, CA, USA, 118–122. doi:10.1145/3643991.3644884
- [50] Yutian Tang, Zhijie Liu, Zhichao Zhou, and Xiapu Luo. 2024. ChatGPT vs SBST: A Comparative Assessment of Unit Test Suite Generation. *IEEE Transactions on Software Engineering* 50, 6 (2024), 1340–1359. doi:10.1109/TSE.2024.3382365
- [51] Michele Tufano, Dawn Drain, Alexey Svyatkovskiy, Shao Kun Deng, and Neel Sundaresan. 2021. Unit Test Case Generation with Transformers and Focal Context. arXiv:2009.05617 [cs.SE] doi:10.48550/arXiv.2009.05617
- [52] Wenhan Wang, Chenyuan Yang, Zhijie Wang, Yuheng Huang, Zhaoyang Chu, Da Song, Lingming Zhang, An Ran Chen, and Lei Ma. 2025. TestEval: Benchmarking Large Language Models for Test Case Generation. In *Findings of the Association for Computational Linguistics: NAACL 2025*, Luis Chiruzzo, Alan Ritter, and Lu Wang (Eds.). Association for Computational Linguistics, Albuquerque, New Mexico, 3547–3562. doi:10.18653/v1/2025.findings-naacl.197
- [53] Cody Watson, Michele Tufano, Kevin Moran, Gabriele Bavota, and Denys Poshyvanyk. 2020. On learning meaningful assert statements for unit test cases. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering* (Seoul, South Korea) (ICSE '20). Association for Computing Machinery, New York, NY, USA, 1398–1409. doi:10.1145/3377811.3380429
- [54] Ratnadira Widyasari, Sheng Qin Sim, Camellia Lok, Haodi Qi, Jack Phan, Qijin Tay, Constance Tan, Fiona Wee, Jodie Ethelda Tan, Yuheng Yieh, Brian Goh, Ferdian Thung, Hong Jin Kang, Thong Hoang, David Lo, and Eng Lieh Ouh. 2020. BugsInPy: a database of existing bugs in Python programs to enable controlled testing and debugging studies. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering* (Virtual Event, USA) (ESEC/FSE 2020). Association for Computing Machinery, New York, NY, USA, 1556–1560. doi:10.1145/3368089.3417943
- [55] xAI. 2025. Grok-3 Model Documentation. <https://docs.x.ai/docs/models/grok-3>. Accessed: 2025-08-19.
- [56] xAI. 2025. Grok-4 Model Documentation. <https://docs.x.ai/docs/models/grok-4>. Accessed: 2025-08-19.
- [57] Lin Yang, Chen Yang, Shutao Gao, Weijing Wang, Bo Wang, Qihao Zhu, Xiao Chu, Jianyi Zhou, Guangtai Liang, Qianxiang Wang, and Junjie Chen. 2024. On the Evaluation of Large Language Models in Unit Test Generation. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering* (Sacramento, CA, USA) (ASE '24). Association for Computing Machinery, New York, NY, USA, 1607–1619. doi:10.1145/3691620.3695529
- [58] Zhiqiang Yuan, Mingwei Liu, Shiji Ding, Kaixin Wang, Yixuan Chen, Xin Peng, and Yiling Lou. 2024. Evaluating and Improving ChatGPT for Unit Test Generation. *Proc. ACM Softw. Eng.* 1, FSE, Article 76 (July 2024), 24 pages. doi:10.1145/3660783

- [59] Qunjun Zhang, Chunrong Fang, Siqu Gu, Ye Shang, Zhenyu Chen, and Liang Xiao. 2025. Large Language Models for Unit Testing: A Systematic Literature Review. arXiv:2506.15227 [cs.SE] doi:10.48550/arXiv.2506.15227
- [60] Qunjun Zhang, Ye Shang, Chunrong Fang, Siqu Gu, Jianyi Zhou, and Zhenyu Chen. 2024. TestBench: Evaluating Class-Level Test Case Generation Capability of Large Language Models. arXiv:2409.17561 [cs.SE] doi:10.48550/arXiv.2409.17561
- [61] Junda Zhao, Shurui Zhou, and Eldan Cohen. 2026. Replication Package for “Do Coverage and Mutation Scores of LLM-Generated Test Suites Correlate With Their Effectiveness? (Replicability Study)”. https://github.com/drixs2050/Cov_mut_bug_detect_correlation. doi:10.5281/zenodo.21437945

Received 2026-01-30; accepted 2026-06-25